BASED ON PYTHON LANGUAGE

5WEEKS INTERNSHIP IN ARCETRI OBSERVATORY

Image processing

BY ALEXIS CARBILLET

Acknowledgments

Before developing this professional experience, it seems important to me to start this internship report by thanking the people without whom this internship could not have taken place under such conditions.

I first thank my uncle, Mr. Marcel CARBILLET, for helping me find an internship through his contacts. Without him, it would have been impossible for me to find an internship abroad.

Then, I thank Mrs. Maria Sofia RANDICH for having accepted my internship request despite the fact that its duration is short.

Then, I particularly want to thank Mr. Armando RICCARDI, my internship tutor who trained me with a lot of pedagogy in adaptive optics and who welcomed me warmly.

Finally, I thank the other members of the Adaptive Optics Group at Arcetri Observatory for their kind welcome and the advice they gave me during these five weeks.

Summary

- Introduction P3
- Choice of sensors P4
- Choice of lens P7
- Define the radius of the pixel circle to use in order to study a star P8
- Camera manipulation P9
- Conclusion P12
- Annexe 1: Part of the table used to select the sensor P13
- Annexe 2: Code in Python language used to determine the radius of the pixel circle to use to study a star P14
- Annexe 3: Code in Python language used to process manipulation data P17
- Annexe 4: Code in Python language used to determine read noise and dark current P26

Introduction

Let’s recall the description of the internship:

The objective of this internship is to discover the working atmosphere of the laboratory and gain experience abroad. The use of English will be necessary to communicate and learning the basics of the Italian language will be interesting. In addition, the purpose of this internship is to develop my skills in optics and computer science through various manipulations on measuring instruments.

What is adaptive optics?

When the rays of a star cross the atmosphere, they deform following the spatial and temporal variation of the refractive index. Instead of having a precise image of the star with a telescope, there is a big bright spot. If two stars are close enough, they can be merged into one. To overcome this lack of resolution, it is necessary to reproduce the inverse effect of the deformations of the wave. For this, a mirror equipped with several thousand actuators is used. It deforms at the frequency of 1kHz. There is also a servo loop between the measuring devices where the sensor and the deformable mirror are located.

How to see the pale stars?

It is necessary to have enough photons to see them. We focus on a bright star that is nearby to use the photons it emits. In the case where there is none, we look for a source of sodium like comets' streaks. Then, with a laser, we point this source, which will allow the emission of photons. Thus is created an "artificial" star.

How to center the image of the Very Large Telescope?

There is an acquisition camera that uses blue light. Depending on the photons landing on the pixels, it is possible to determine the position of the target object on the image. The module with the instruments then moves until the target star is in the center of the image. A wider spectral range including green light will be discussed later. It has been chosen to include it because it is the light for which the human eye is the most sensitive. My goal was to choose the new camera for this work.

What is the astropy library ?

Astropy is a collection of software packages written in the Python programming language and designed for use in astronomy.

Choice of sensors

Before starting, some definitions are needed for a better understanding:

- dark current: signal detected when there is no light, this is its own radiation
- quantum efficiency: ratio between the number of electronic charges collected and the number of photons incident on a reactive photo surface
- binning: grouping information of several pixels into one, which increases the sensitivity. However the accuracy decreases. There are two types: A first that is in vertical or horizontal line, a second in two dimensions (for example a pixel groups the data acquired in a square of 4 pixels). It helps to act on digital noise
- plate scale: connection between the angular separation of an object with the linear separation of its image in the focal plane
- ADU (Analog-Digital Units) : number in binary writing

There is a constant presence of noise that limits the accuracy of the image and can hide stars that do not have enough brightness. It comes from three sources:

- Thermal noise determined by the product of dark current ($\acute{N}_d$) and time(t)
- Digital noise determined by the product of the number of incoming photons per second ($\acute{N}_{ph}$, time (t) and quantum efficiency (QE)
- Noise of reading by the device (standard deviation, σ) with p the number of pixels

The standard deviation of noise is as follows:

$$\sqrt{\left(QE \times \acute{N}_{ph} + \acute{N}_d \times p\right) \times t + \sigma^2 \times p}$$

The signal received is: $QE \times \acute{N}_{ph} \times t$

The SNR is the ratio of the two. It was set at 3. We then have an equation, in which we will isolate the time to determine which GigE interface (Gigabit Ethernet) camera is the most efficient. We then obtain a second degree equation in t:

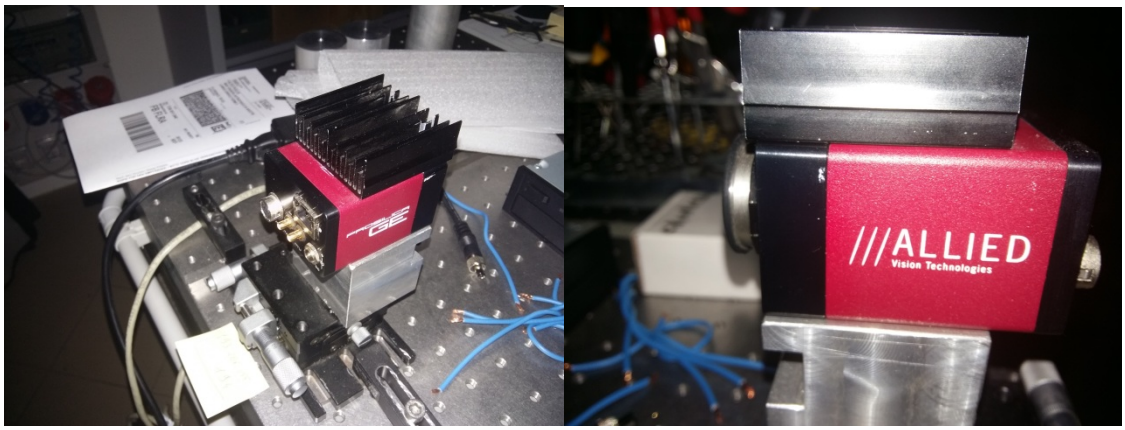
$$0 = \acute{N}_{ph}^{\ 2} \times QE^2 \times t^2 - \left(\acute{N}_d \times p + \acute{N}_{ph} \times QE\right) \times SNR^2 \times t - SNR^2 \times \sigma^2 \times p$$

We choose to keep the highest solution:

$$t \quad \frac{\left(\acute{N}_d \times p + \acute{N}_{ph} \times QE\right) \times SNR^2 + \sqrt{\left(\left(\acute{N}_d \times p + \acute{N}_{ph} \times QE\right) SNR^2\right)^2 - 4\,\acute{N}_{ph}^{\ 2} QE^2 SNR^2 \sigma^2 p}}{2 \times \acute{N}_{ph}^{\ 2} \times QE^2}$$

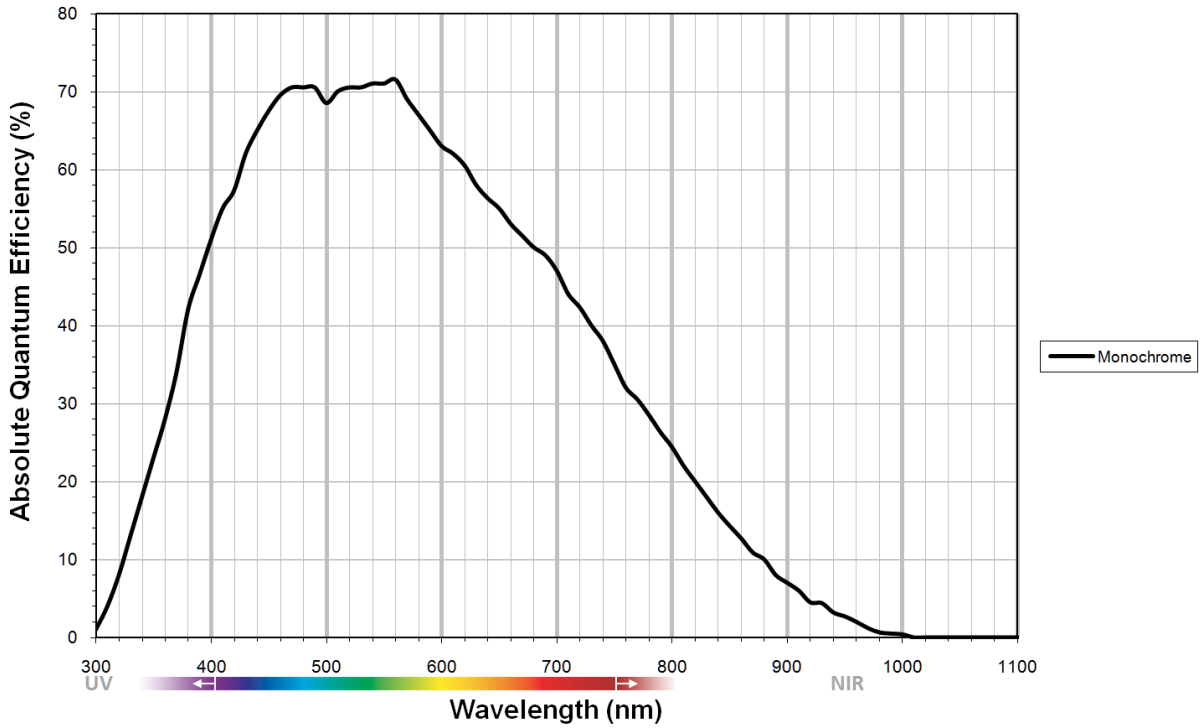
For the choice of the camera, I used the following catalog that I had to re-read whenever my tutor asked me to look for new information:
https://www.alliedvision.com/en/products/cameras.html#interfacefilter%2F3%2F

They already had a camera, equipped with ON Semi sensor KAI-04022. Only, it no longer meets the performance criteria, hence my intervention in the search for a new camera. You can see it on the photos below:



To define quantum efficiency, I averaged its value for wavelengths between 400nm and 590nm. So I have not treated those that have infrared application range.

Example of a quantum efficiency graph as a function of the wavelength of the ray incident on the sensor



Why this range of values?

Most stars are red (except those who are young and blue). Blue light is used for this camera and the rest is used for data collection that will allow the creation of the image.

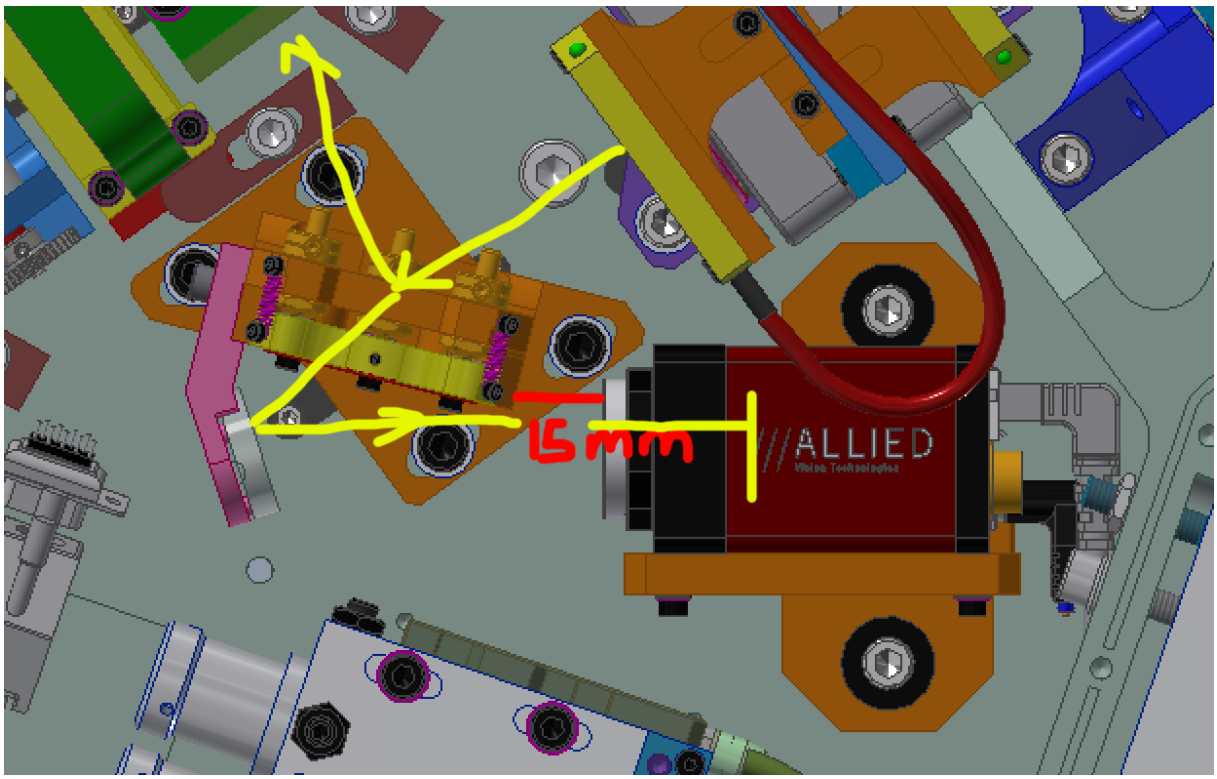
I applied a criterion on the size of the sensor for a given magnification to avoid that some light rays are not treated because of a sensor too small. For that, I multiplied the height of the sensor by the flat scale and I checked that this is superior to 12 seconds of arc (arcsec), criterion required. I then looked for different information such as dark current and read-out noise in different documents such as their instructions or comparative tables between different sensors. Since dark current information is difficult to find, 14 sensors could not be processed later. In addition, it varies greatly with temperature. It was necessary to check that the ambient temperature during its measurement is between 5 and 20 ° C. Then, I looked for the most important possible binning per sensor because it reduces the digital noise so it influences the time needed to process the information.

Finally, I tried larger scale scales. The one previously considered was $\frac{1}{770}$ arcsec/µm. I tried multiplying this value by 2, 3 and 4. The distance between the mirror and the camera will always be the same. However, in order to shrink the image and thus be able to use a smaller sensor, it is possible to add between these two elements a diverging lens that would allow the outgoing rays to be parallel to the axis (image focus on the object) and a convergent lens whose focus image would be on the camera (see following drawing in the next page). I calculated which scale scale would be needed for each camera that does not have a large enough pixel count. With the initial flat scale, the most powerful sensor that can be chosen is the Sony IMX304, which corresponds to the Manta G-1236 camera (see photo below):

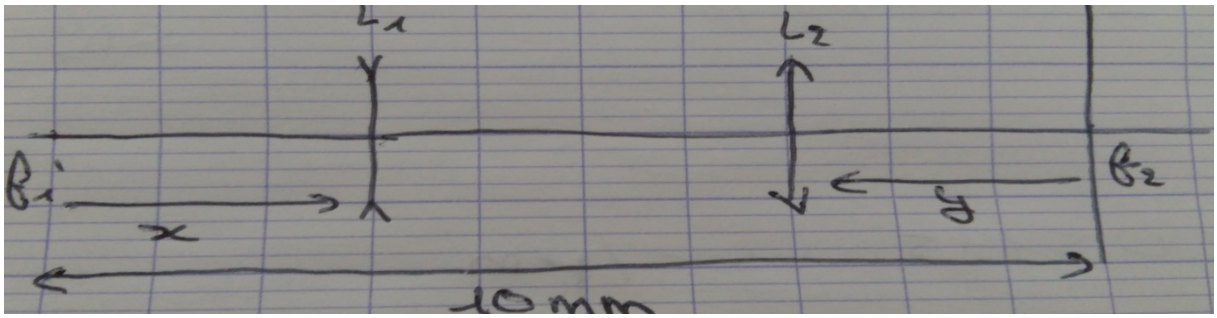
Applying binning, the most effective sensor is the Sony ICX814, present in the Manta G-917 and Prosilica GT 3400 cameras (see photos below):

With the application of binning and the use of appropriate lenses, the most effective sensor is the Sony ICX414 present in the cameras Manta G-033 and Prosilica GC 655, see the photos below:

Choice of lens



The required diameter of the lenses will depend on their positions, as well as their vergences. We consider here that M= $\frac{1}{770}$ arcsec/ µm



Using Thales theorem, we find that the radius R1 of the diverging lens must be greater than or equal to: $\frac{4{,}62 \times x}{2 \times 0{,}1}$ (The 4,62mm comes from the minimum diameter of the sensor needed to gather all the information).

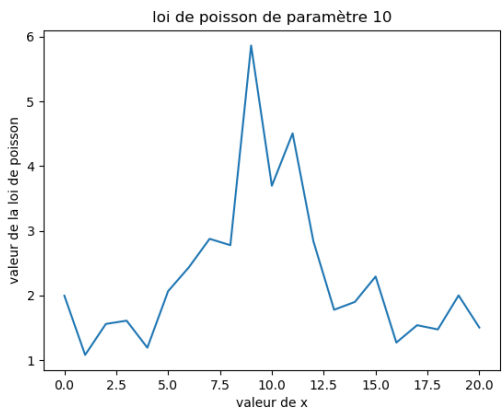
By the fact that the rays coming out of L1 are parallel to the axis, the radius R2 will be equal to R1 or higher if we take precautions (the rays will surely not be perfectly parallel because of the errors that may occur during editing).

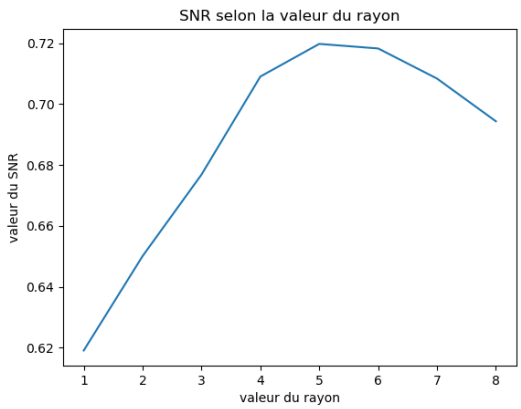
I have made the assumption that the lenses are respectively vergence V1 and V2, they each have a thickness of 1mm and it takes at least 0.1 mm focal length. So (x,y)€[0,1mm;8,9mm]^2. Vergence is the opposite of the focal length, so we have (V_1,V_2)€[112δ;1000δ]^2.

Define the radius of the pixel circle to use in order to study a star

When the camera sends a signal, it is noisy for the reasons previously seen. In order to know if we see a star or not, it is necessary to impose a criterion. In general, a criterion of 3 times the value of the apparent plateau in the graph of the number of photons received per pixel is chosen.

One can also define the radius of the pixel circle to use to study a star by plotting the graph showing the SNR as a function of the radius used (in pixels). The circle will be centered on the pixel receiving the maximum of photons. The radius chosen will be that corresponding to the maximum of SNR. This is what I did in the code written in Python which is in Annexe 2. We can see the graphs obtained by having followed a parameter Poisson's law $\mu=10$ multiplied by a coefficient 10, of level 1 and with 21 points taken.

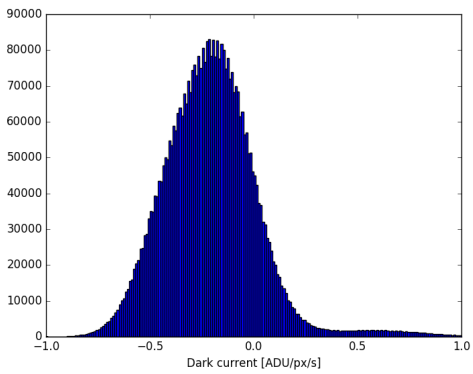


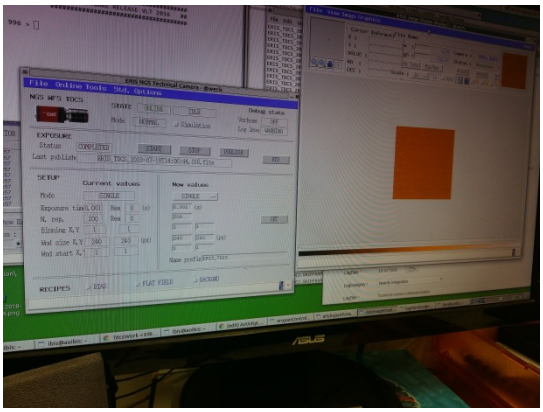


In my simulation, the pixel receiving the maximum of photons was the number 9 out of 21. The maximum radius is 8 pixels for the circle centered on it. We find that the optimal radius is 5pixels. I treated the problem in one dimension even if a circle is in two dimensions because the SNR, ratio between the signal and the noise of the pixels contained in a circle is the same as on only one of its rays. The determined radius depends on the input signal. The function I created takes into account this fact. The result therefore adapts to this phenomenon.

Camera manipulation

The problem they have with the measurements they have made through the camera they own is that they find a dark current per pixel negative, which is physically impossible. It was -0.11 ADU / s / px. The track that was proposed to me is that the first measurements were made on a time interval too small so the camera heated, so that it had more noise than a longer time interval. However, being in the dark, the noise should accumulate over time so, become more and more important. The data and graphics were processed in Python and I had to use my computer skills in order to appropriate the principles of the algorithm. We can find this code, provided by Mr. Lorenzo BUSONI, that I commented in Annexe 3.





Next, we can see the distribution of the average number of pixels on the different integration times as a function of the dark current that was obtained.

I made different shots with the camera in the dark. We first took 100 with an integration time of 1ms in order to determine the dark current. Then we took another 200 to compare the results with the first 100. You can see the control screen used in the photo on the right to capture these 200 images. Then, we have taken 200 for integration times of 1ms, 10ms, 100ms, 500ms, 1s and 2s. I created an algorithm that processes the data. A first for the first 100 images, a second for the next 200 and finally a last one that treats the remaining ones. They are grouped into 3 different files, each containing its corresponding algorithm. To use my algorithms, I had to rename the images because their name had no link to open them one by one in a loop. They make it possible to calculate the average values and temporal variances per image pair for the same integration time. The spatial averages (on all the pixels of the image) of these temporal averages and variances are also calculated. The fact of calculating the temporal variance by a pair of successive images and then averaging them makes it possible to keep the non-correlation between the data of the different images, which is necessary in order to be able to use the formula making it possible to find the reading noise. and the gain (see below).

Definition of the time average for a given integration time and for a pixel :

$$f_{i,j}^{m} = \frac{2}{N_m} \times \sum_{k=1,2,3,\ldots,N_{m-1}} (F_{i,j,k+1}^{m} - F_{i,j,k}^{m})$$

Definition of the spatial mean of the temporal averages : $< f^m >= \frac{1}{N_x \times N_y} \times \sum_{i,j} f_{i,j}^{m}$

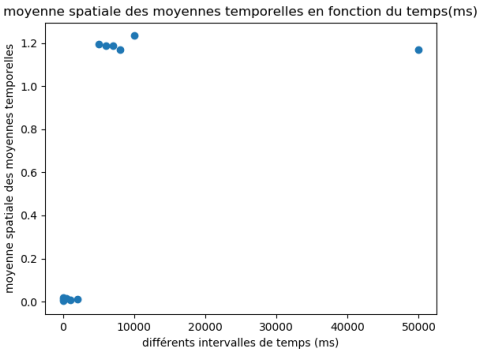
Definition of the variance between 2 values of a pixel : $V_s = \left(S_1 - S_2\right)^2$

But we also have: $V_s = 2G\overline{S'} + 2\sigma_B^2$ with : $\overline{S'} = \frac{1}{2}\left\{\overline{(S_1 - B)} + \overline{(S_2 - B)}\right\}$

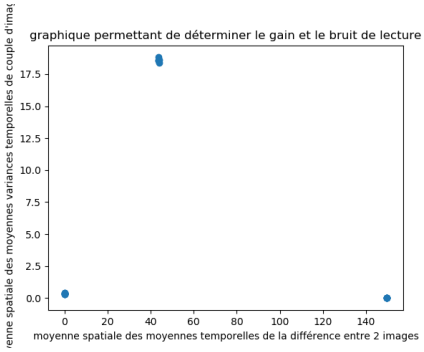
And :

- m : integration time
- i : row number of the pixel
- j : column number of the pixel
- k : number of the image for an integration time m
- N_m : number of image couples for an integration time m
- f : temporal mean of the pixel value (ADU)
- G : camera gain (ADU/e^-)
- S_1 : pixel value of the first image of the couple (ADU)
- S_2 : pixel value of the second image of the couple (ADU)
- B : noise for the weakest integration time (ADU)
- σ_B : standard deviation of reading noise $(ADU^2/e^-)^{1/2}$

The first measurements prove to be unsatisfactory because to read the reading noise, the only noise not depending on the integration time, it is necessary to process on a panel of more important integration time and to add external light in order to increase the variance and to have a more significant curve. Similarly, to determine the dark current, it is necessary to have a longer integration time panel. The possible measurement range is 75 µs to 53.7 s. So I decided to take additional measurements in the dark with time intervals of 10s and 50s. For reading noise, I measured with an integration time of 1ms and different levels of incoming photons in the lens of the camera by letting the outside light rays enter the camera lens. It was therefore necessary to use the algorithm previously used for the dark current by processing the measurements they were dedicated to it. Only the graph plot changes.



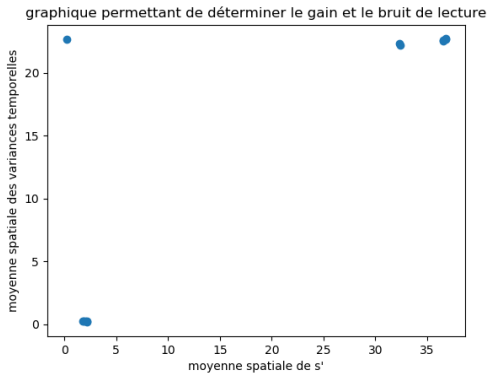
After taking the extra measurements for the dark current, I noticed that there are two trends and that the transition from one to the other is between the integration times of 2s and 10s. So, it is necessary to take additional measures with integration of 5s, 6s, 7s and 8s in order to be able to transition less. All images taken so far by 240. See opposite the graph obtained.

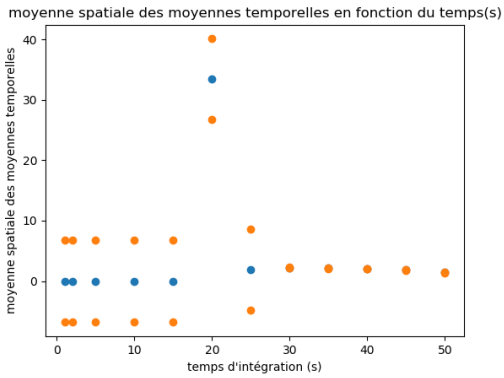


Following the analysis of these results, the difference to overcome was due to the fact that the first measurements were made with only a cache in front of the lens of the camera because there was nobody so the lights were off (low level) . While for the following, I added curtains all around the table where the device was located because there were other people in the laboratory and they needed to turn on the lights

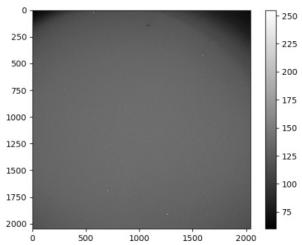
(landing high). The rightmost points are due to a problem of data encoding by the acquisition software. The data was coded only on 1 byte which did not allow to reach values higher than 255 while it is possible with this camera. People in the lab had to reprogram it to double bytes. So I made another measurement, taking images of size 2048 by 2040 which corresponds to the maximum size allowed by the device. Thus, 10 images per integration time were sufficient.

Then, I took the integration times of 1, 2, 5, 10, 15, 20, 25, 30, 35, 40, 45 and 50s to measure the dark current in addition to the groups of images taken with different levels of brightness. I also added the 99.7% confidence intervals by considering a Gaussian curve for measurements made as part of the dark current determination. I reused this data for reading noise determination. Here are the graphs obtained:





It can be seen that for the second graph, from an integration time of 30s, the 99.7% confidence interval of the pixel values decreases sharply. In addition, I think the value for the integration time of 20s is wrong. The following code, in python language, allowed me to visualize the images taken:



```
import astropy.io.fits
import matplotlib.pyplot as plt
plt.imshow(astropy.io.fits.getdata('86.fits'), cmap='gray') # image 86 for exemple
plt.colorbar()
plt.show()
```

Conclusion

We have previously seen the basics of adaptive optics as well as the main sources that generate noise when taking a measurement by a camera. I also searched for a camera faster than the one they currently have to their provisions and that meets the criteria requested. This is the Sony ICX414 sensor, fitted to the Manta G-1236 camera, for a $\frac{1}{770}$ arcsec/µm (See annexe 1).

Then, I determined a range of diameter and vergence for a two-lens image shrinkage system. They must have a vergence between 112d and 1000d and a radius between 2.3mm and 205.6mm. I also did a simulation of a light signal reaching the camera and determined the radius needed to be able to have the best possible signal quality (see algorithm annexe 2).

Then, I studied the algorithm in Python language provided by Mr. Lorenzo BUSONI that allows to process the data acquired by the camera that the group has (see annexe 3). After having taken some pictures with the camera myself, I created my own algorithms in Python language (annexe 4) in order to process the information provided by them and I renamed them. There were 4240 all measures combined.

In addition to technical skills, I also developed my language skills, such as speaking in English or listening in Italian. Having a very good social climate, this research laboratory allowed me to spend a very pleasant internship and rich in personal development.

Finally, this internship allowed me to discover the working method used in this laboratory and which, I think, is used in many others. I realized the importance of checking a sample of results before processing all the data to avoid working with them if they are wrong or discovering as soon as possible the technical problems.

Annexe 1 : Part of the table used to select the sensor

| Paramètres | | |

Annexe 2 : Code in Python language used to determine the radius of the pixel circle to use to study a star

```
# import libraries
import matplotlib.pyplot as plt
import numpy as np
import random
import math

def poisson (mu,k): # amplified by a 10 coefficient
	return 10*mu**k*np.exp(-mu)/math.factorial(k)
def dark_noise(n): # n: number of points
	a=[]
	for i in range(n):
		a.append(random.random()) # create noise
	return a		# return list which contains the noise
def readout_noise(n,f,mu,p): # p the landing
	a=[]
	for i in range(n):
		if(f(mu,i)>p): # photons arrive on the pixel
			a.append(3*random.random()) # readout_noise
		else:
			a.append(0) # no noise because there are no photons arriving on these
		pixels
	return a			# returns the list containing the photon reading noise
# we trace the signal that would be sent by the camera
def trace(n,f,mu,p):
	x=[]
	y=[]
	for i in range(n):
		x.append(i)  # list of abscissa, they are integers
		y.append(poisson(mu,i)+p+dark_noise(n)[i]+readout_noise(n,f,mu,p)[i])
		# noisy signal
	plt.plot(x,y)		# graph creation
	plt.xlabel('x value')	# abscissa axis legend
	plt.ylabel('value of the fish law')	# ordinate axis legend
	plt.title('fish law of parameter 10 noisy')		# graphic title
	plt.show()			# plot graphic
```

```
trace(21,poisson,10,1) # we have only 21 points, execution of the trace function

def trace2(n,f,mu,p):   # SNR trace according to the pixel interval around the most receptive
        x=[]
        y=[]
        bruit=[]
        signal=[]
        for i in range(n):
                x.append(i)        # list of abscissa, they are integers
                y.append(poisson(mu,i)+p+dark_noise(n)[i]+readout_noise(n,f,mu,p)[i])
                bruit.append(dark_noise(n)[i]+readout_noise(n,f,mu,p)[i])     # noisy signal
                signal.append(poisson(mu,i))
        m=max(signal)       # look for the signal peak (maximum of the ordinates)
        k=signal.index(m)   # look for the pixel that receives the maximum photon (the most
        receptive)
        signal_total=0      # SNR numerator
        bruit_total=0       # SNR denominator
        SNR=[]
        rayon=[]
        for r in range(1,min(n-k,k)): # look at the pixels located in the circle of radius r and the
        one that receives the maximum number of photons
                for j in range(k-r,k+r): # look at the noise around this pixel for a given radius
                        signal_total+=signal[j]   # value addition
                        bruit_total+=bruit[j]     # value addition
                SNR.append(signal_total/bruit_total)  # divide by the corresponding noise
                rayon.append(r)
        plt.plot(rayon,SNR)    # graphic creation
        plt.xlabel('radius value')      # abscissa axis legend
        plt.ylabel('SNR value')                 # ordinate axis legend
        plt.title('SNR according to the value of the radius')     # graphic title
        plt.show()          # plot graphic


trace2(21,poisson,10,1)  # executes the trace2 function with 21 points, centered on 10
```

Annexe 3 : Code in Python language used to process manipulation data

```
# import libraries
import os
import numpy as np
import pyfits
from matplotlib.pyplot import plot, legend, errorbar, xlim, xlabel, ylabel, title, figure, imshow,
colorbar, clf, ylim
import matplotlib.pyplot as plt
from matplotlib import cm

class MeasureAVTCamera():
	def __init__(self, camera):
		self._cam= camera # initialization cam

	# allows the transition from 2D to 3D of the class camera
	def _getCube(self, nFrames):
		a= [] # creation list a
		for _ in range(nFrames): # nFrames images will be taken by the camera
			a.append(self._cam.read()) # saves the image data
		arr=np.dstack([x.toNumpyArray() for x in a]) # transition to 3D
		return arr

	@staticmethod
	def getCube(nFrames, avtCamera): # allows the transition from 2D to 3D of the class
	camera
		a= [] # creation list a
		for _ in range(nFrames): # nFrames images will be taken by the camera
			a.append(avtCamera.readFrame())# saves the image data
		arr=np.dstack([x for x in a]) # transition to 3D
		return arr

	# allows you to define an average of the measurements
	def getBias(self):
		self._cam.setExposureTime(0.1) # for an integration time of 0.1ms
		return np.mean(self._getCube(10), axis=2) # return the mean
```

```
# acquisition of camera measurements without brightness
def getDarkFrames(self):
        darkExposureTime= np.logspace(-1, 4, 13)
        counter= 0                    # counter
        for expTime in darkExposureTime:
                self._cam.setExposureTime(expTime)
                frames= self._getCube(10) # 3D passage of 10 images
                hdr= pyfits.Header()
                hdr.append(('expms', expTime))
                hdr.append(('binning', 1)) # the binning is 1 (no grouping)
                filename= '/home/pygo/tmp/avt/dark/dark-%d.fits' % counter
                # path to the file
                pyfits.writeto(filename, frames, hdr, clobber=True) # write
                counter+= 1                # the counter is incremented

# determination of the camera gain
def analyzeGain(self):
        folder= '/home/pygo/tmp/avt/gain2' # path to the file
        exptimes= np.array([1, 2, 5, 10, 20, 50, 100, 200, 500])*1000
        med= []  # creation of the list of medians
        var= []     # creation of the variance list
        for exptime in exptimes:
                filename= os.path.join(folder, "frames%d.fits" % exptime)
                print(filename)
                cube= pyfits.getdata(filename) # gets the data from the file
                med.append(np.median(cube, axis=2))  # add median
                var.append(np.std(cube, axis=2)**2) # add variance
        return {'median': np.dstack([x for x in med]), 'variance': np.dstack([x for x in
        var]), 'expTimes': exptimes} # returns concatenated matrices of medians and
        variances

# concatenates arrays of the means and variances of all measurements
def analyzeDark(self):
        folder= '/home/pygo/tmp/avt/dark2' # path to the file
        exptimes= np.array([0.1, 0.2, 0.5, 1, 2, 5, 10, 20, 50, 100, 200, 500,
        1000])*1000 # list of time intervals
        med= []   # creation of the list of medians
        var= []     # creation of the list of variances
        for exptime in exptimes: # below is the path to the file
                filename= os.path.join(folder, "frames%d.fits" % exptime)
                print(filename)
                cube= pyfits.getdata(filename)
                med.append(np.median(cube, axis=2)) # add median
                var.append(np.std(cube, axis=2)**2) # add variance
        return {'median': np.dstack([x for x in med]), 'variance': np.dstack([x for x in
        var]), 'expTimes': exptimes} # returns concatenated matrices of medians and
        variances
```

```
# allows to draw a graph in the form of a histogram
def plotBiasHisto(self, biasImage):
	import matplotlib.pyplot as plt # import library
	biasm= np.median(biasImage, axis=2) # median according to the 3rd
	dimension
	n, bins, patches= plt.hist(biasm.flatten(), 15, range=(30, 45)) # histogramme
	creation

# linear regression of the gain curve (y = mx + c)
def fitGain(self, mediana, variance):
	m= np.zeros(mediana.shape[0:2])
	c= np.zeros(mediana.shape[0:2])
	for xi in range(2048):
		print(xi) # print from 0 to 2047
	for yi in range(2048):
		x= mediana[xi, yi, :] # median according to the 3rd dimension
		y= variance[xi, yi, :] # variance according to the 3rd dimension
		A= np.vstack([x, np.ones(len(x))]).T
		m[xi, yi], c[xi, yi]= np.linalg.lstsq(A, y)[0]
	return m, c	# returns the reference coefficient and the y-axis at the origin

def fitDark(self, expTimeDarkSec, medianDark, bias, gain):
	dark= gain * (medianDark- bias.reshape(2048, 2048, 1)) # calcul darkness
	y= dark.reshape(2048**2, 13).T
	A= np.vstack([expTimeDarkSec, np.ones(len(expTimeDarkSec))]).T
	m, c= np.linalg.lstsq(A, y)[0] # coefficients of the linear regression
	darkMap= m.reshape(2048, 2048) # order matrix 2048
	#n, bins, patches= plt.hist(darkMap.flatten(), 200, range=(0,20)) #
	histogramme creation
	return darkMap	# return matrix
```

```
# allows the calculation of the dark current
@staticmethod
def darkCurrent(pygo): # list of integration times below
        times=[0.1,0.5,1.0,5.,10.,50.,100,300,500,600,1000,5000,7000,10000]
        times+=times+times[::-1]+times[::-1]
        centerGau=[]          # list of Gauss curve centers
        pygo.auxCamera.setExposureTime(0.1) # for 0.1ms
        fra= pygo.auxCamera.getFutureFrames(50, timeoutSec=100).toNumpyArray()
        bias= np.median(fra, axis=2) # median according to the 3rd dimension
        bins=np.linspace(-20, 20, 201)
        fitfunc= lambda p, x: p[0]*np.exp(-(x-p[1])**2/(2*p[2]**2))
        errfunc= lambda p, x, y: fitfunc(p,x) - y
        p0=[1000.,-1.,1.]
        from scipy import optimize # import library
        for t in times:
                pygo.auxCamera.setExposureTime(t) # for a t time
                fra= pygo.auxCamera.getFutureFrames(10,
                timeoutSec=1000).toNumpyArray()
                dark= (np.median(fra, axis=2) - bias)  # calculation of darkness
                hist, bins2= np.histogram(dark.flatten(), bins) # histogramme creation
                p1,suc= optimize.leastsq(errfunc, p0[:], args=(bins[:1]+np.diff(bins)
                [0]/2.,hist))
                centerGau.append(p1[1]) # add Gauss curve centers
        return times, centerGau  # returns the list of times and centers of Gauss
        curves

# measures and traces the histogram of the results obtained in the dark
def dark(self, pygo):
        import pyfits # import librairies
        import os
        pygo.auxCamera.setExposureTime(.1) # for 0.1ms
        fra= pygo.auxCamera.getFutureFrames(100,timeoutSec=100).toNumpyArray()
        bias=np.median(fra, axis=2)  # median according to the 3rd dimension
        baseDir= '/home/pygo/tmp/avt/dark6' # path to the file
        pyfits.writeto(os.path.join(baseDir, 'bias.fits'), bias) # save image
        biasv=np.std(fra, axis=2) # variance according to the 3rd dimension
        pyfits.writeto(os.path.join(baseDir, 'bias_std.fits'), biasv) # save image




        # measurements performed at different time intervals
        pygo.auxCamera.setExposureTime(0.1, timeoutSec=11.) # for 0.1ms
        fra0= pygo.auxCamera.getFutureFrames(10,timeoutSec=200).toNumpyArray()
        dark0= (np.median(fra0, axis=2) - bias) # calculation of darkness
        pyfits.writeto(os.path.join(baseDir, 'frames0ms.fits'), fra0) # save image
```

```
pygo.auxCamera.setExposureTime(500, timeoutSec=11.) # for 500ms
fra500=pygo.auxCamera.getFutureFrames(10,
timeoutSec=200).toNumpyArray()
dark500= (np.median(fra500, axis=2) - bias) # calculation of darkness
pyfits.writeto(os.path.join(baseDir, 'frames500ms.fits'), fra500)
# save
pygo.auxCamera.setExposureTime(1000, timeoutSec=11.) # for 1000ms
fra1000= pygo.auxCamera.getFutureFrames(10,
timeoutSec=200).toNumpyArray()
dark1000= (np.median(fra1000, axis=2) - bias) # calculation of darkness
pyfits.writeto(os.path.join(baseDir, 'frames1000ms.fits'), fra1000)
# save
pygo.auxCamera.setExposureTime(2000, timeoutSec=11.) # for 2000ms
fra2000= pygo.auxCamera.getFutureFrames(10,
timeoutSec=200).toNumpyArray()
dark2000= (np.median(fra2000, axis=2) - bias) # calculation of darkness
pyfits.writeto(os.path.join(baseDir, 'frames2000ms.fits'), fra2000)
# save
pygo.auxCamera.setExposureTime(5000, timeoutSec=11.) # for 5000ms
fra5000= pygo.auxCamera.getFutureFrames(10,
timeoutSec=200).toNumpyArray()
dark5000= (np.median(fra5000, axis=2) - bias) # calculation of darkness
pyfits.writeto(os.path.join(baseDir, 'frames5000ms.fits'), fra5000)
# save
pygo.auxCamera.setExposureTime(10000, timeoutSec=11.) # for 10000ms
fra10000= pygo.auxCamera.getFutureFrames(10,
timeoutSec=200).toNumpyArray()
dark10000= (np.median(fra10000, axis=2) - bias) # calculation of darkness
pyfits.writeto(os.path.join(baseDir, 'frames10000ms.fits'), fra10000)
# save

expTimeSec= np.array([0, 500, 1000, 2000, 5000, 10000]) / 1000.
dark=np.dstack([dark0, dark500, dark1000, dark2000, dark5000, dark10000])
# list of different darkness recorded for different integration times
A= np.vstack([expTimeSec, np.ones(len(expTimeSec))]).T
y= dark.reshape(2048**2, dark.shape[2]).T
m, c= np.linalg.lstsq(A, y)[0] # coefficients of the linear regression
darkMap= m.reshape(2048, 2048) # order matrix 2048
pyfits.writeto(os.path.join(baseDir, 'darkMap.fits'), darkMap) # save

n, bins, patches= plt.hist(darkMap.flatten(), 200, range=(0,20)) # histogramme
creation
imshow(darkMap, vmin=0, vmax=2)
```

```
# allows to calculate the error made by the instrument during its measurement
    def readOutNoise(self, pygo):
        import pyfits            # import librairies
        import os
        GAIN_E_PER_ADU= 5 # number of electrons per pixel
        pygo.auxCamera.setBinning(1) # binning of 1 (there is no grouping)
        pygo.auxCamera.setExposureTime(.1) # for 0.1ms
        fra= pygo.auxCamera.getFutureFrames(20, timeoutSec=100).toNumpyArray()
        bias=np.median(fra, axis=2) # median according to the 3rd dimension
        baseDir= '/home/prisma/tmp/avt/ron' # path to the file
        pyfits.writeto(os.path.join(baseDir, 'bias_bin1.fits'), bias) # write the mediane
        biasv=np.std(fra, axis=2) # variance according to the 3rd dimension
        pyfits.writeto(os.path.join(baseDir, 'bias_bin1_std.fits'), biasv) # write the
        variance
        n, bins, patches= plt.hist(GAIN_E_PER_ADU*biasv.flatten(),\50, range=(0,40),
        histtype='bar', ec='k') # histogramme creation

        pygo.auxCamera.setBinning(8) # binning of 8 pixels
        pygo.auxCamera.setExposureTime(.1) # for 0.1ms
        fra= pygo.auxCamera.getFutureFrames(100, timeoutSec=100).toNumpyArray()
        bias=np.median(fra, axis=2) # median according to the 3rd dimension
        baseDir= '/home/prisma/tmp/avt/ron' # path to the file
        pyfits.writeto(os.path.join(baseDir, 'bias_bin8.fits'), bias) # write the mediane
        biasv=np.std(fra, axis=2) # variance according to the 3rd dimension
        pyfits.writeto(os.path.join(baseDir, 'bias_bin8_std.fits'), biasv) # write the
        variance
        n, bins, patches= plt.hist(GAIN_E_PER_ADU*biasv.flatten(),\50, range=(0,40),
        histtype='bar', ec='k') # histogramme creation

    # determination of the dark current
    def darkCurrent20170517(self, pygo):
        import pyfits            # import librairies
        import os
        GAIN_E_PER_ADU= 5 # number of electrons per pixel
        pygo.auxCamera.setBinning(1) # binning of 1 (there is no grouping)
        pygo.auxCamera.setExposureTime(.1) # for 0.1ms
        fra= pygo.auxCamera.getFutureFrames(20, timeoutSec=100).toNumpyArray()
        bias=np.median(fra, axis=2) # median according to the 3rd dimension
        baseDir= '/home/prisma/tmp/avt/dark_current' # path to the file
        pyfits.writeto(os.path.join(baseDir, 'dark_01ms.fits'), bias) # write
        biasv=np.std(fra, axis=2) # variance according to the 3rd dimension et l'écris
        pyfits.writeto(os.path.join(baseDir, 'dark_01ms_std.fits'), biasv)
        n, bins, patches=
        plt.hist(GAIN_E_PER_ADU*biasv.flatten(),\50,range=(0,40),histtype='bar',
        ec='k') # histogramme creation
```

```
pygo.auxCamera.setExposureTime(100) # for 100ms
fra= pygo.auxCamera.getFutureFrames(20, timeoutSec=100).toNumpyArray()
bias100ms=np.median(fra, axis=2) # median according to the 3rd dimension
baseDir= '/home/prisma/tmp/avt/dark_current' # path to the file
pyfits.writeto(os.path.join(baseDir, 'dark_100ms.fits'), bias100ms) # write
biasv100ms=np.std(fra, axis=2) # variance according to the 3rd dimension
pyfits.writeto(os.path.join(baseDir, 'dark_100ms_std.fits'), biasv100ms)
n, bins, patches= plt.hist(GAIN_E_PER_ADU*biasv100ms.flatten(),\50,
range=(0,40),histtype='bar', ec='k') # histogramme creation

pygo.auxCamera.setExposureTime(1000) # for 1000ms
fra= pygo.auxCamera.getFutureFrames(20, timeoutSec=100).toNumpyArray()
bias1000ms=np.median(fra, axis=2) # median according to the 3rd dimension
baseDir= '/home/prisma/tmp/avt/dark_current' # path to the file
pyfits.writeto(os.path.join(baseDir, 'dark_1000ms.fits'), bias1000ms) # write
biasv1000ms=np.std(fra, axis=2) # variance according to the 3rd dimension
pyfits.writeto(os.path.join(baseDir, 'dark_1000ms_std.fits'), biasv1000ms)
n, bins, patches= plt.hist(GAIN_E_PER_ADU*biasv1000ms.flatten(),\50,
range=(0,40),histtype='bar', ec='k') # histogramme creation

pygo.auxCamera.setExposureTime(10000) # for 10000ms
fra= pygo.auxCamera.getFutureFrames(10, timeoutSec=200).toNumpyArray()
bias10000ms=np.median(fra, axis=2) # median according to the 3rd dimension
baseDir= '/home/prisma/tmp/avt/dark_current' # path to the file
pyfits.writeto(os.path.join(baseDir, 'dark_10000ms.fits'), bias10000ms) #write
biasv10000ms=np.std(fra, axis=2) # variance according to the 3rd dimension
pyfits.writeto(os.path.join(baseDir, 'dark_10000ms_std.fits'), biasv10000ms)
n, bins, patches= plt.hist(GAIN_E_PER_ADU*biasv10000ms.flatten(),\50,
range=(0,40),histtype='bar', ec='k') # histogramme creation

import pyfits            # import librairies
import os
GAIN_E_PER_ADU= 5 # number of electrons per pixel
pygo.auxCamera.setBinning(8) # binning of 8
pygo.auxCamera.setExposureTime(.1) # for 0.1ms
fra= pygo.auxCamera.getFutureFrames(20, timeoutSec=100).toNumpyArray()
bias=np.median(fra, axis=2) # median according to the 3rd dimension
baseDir= '/home/prisma/tmp/avt/dark_current_bin8' # path to the file for a
binning of 8
pyfits.writeto(os.path.join(baseDir, 'dark_01ms.fits'), bias)  # write mediane
biasv=np.std(fra, axis=2) # variance according to the 3rd dimension
pyfits.writeto(os.path.join(baseDir, 'dark_01ms_std.fits'), biasv) # write
n, bins, patches= plt.hist(GAIN_E_PER_ADU*biasv.flatten(),\50,
range=(0,40),histtype='bar', ec='k') # histogramme creation
```

```
pygo.auxCamera.setExposureTime(100) # for 100ms
fra= pygo.auxCamera.getFutureFrames(20, timeoutSec=100).toNumpyArray()
bias100ms=np.median(fra, axis=2) # median according to the 3rd dimension
pyfits.writeto(os.path.join(baseDir, 'dark_100ms.fits'), bias100ms) # write
biasv100ms=np.std(fra, axis=2) # variance according to the 3rd dimension
pyfits.writeto(os.path.join(baseDir, 'dark_100ms_std.fits'), biasv100ms)
n, bins, patches= plt.hist(GAIN_E_PER_ADU*biasv100ms.flatten(),\50,
range=(0,40),histtype='bar', ec='k') # histogramme creation

pygo.auxCamera.setExposureTime(1000) # for 1000ms
fra= pygo.auxCamera.getFutureFrames(20, timeoutSec=100).toNumpyArray()
bias1000ms=np.median(fra, axis=2) # median according to the 3rd dimension
pyfits.writeto(os.path.join(baseDir, 'dark_1000ms.fits'), bias1000ms) # write
biasv1000ms=np.std(fra, axis=2) # variance according to the 3rd dimension
pyfits.writeto(os.path.join(baseDir, 'dark_1000ms_std.fits'), biasv1000ms)
n, bins, patches= plt.hist(GAIN_E_PER_ADU*biasv1000ms.flatten(),\50,
range=(0,40),histtype='bar', ec='k') # histogramme creation

pygo.auxCamera.setExposureTime(5000) # for 5000ms
fra= pygo.auxCamera.getFutureFrames(20, timeoutSec=200).toNumpyArray()
bias5000ms=np.median(fra, axis=2) # median according to the 3rd dimension
pyfits.writeto(os.path.join(baseDir, 'dark_5000ms.fits'), bias5000ms) # write
biasv5000ms=np.std(fra, axis=2) # variance according to the 3rd dimension
pyfits.writeto(os.path.join(baseDir, 'dark_5000ms_std.fits'), biasv5000ms)
n, bins, patches= plt.hist(GAIN_E_PER_ADU*biasv5000ms.flatten(),\50,
range=(0,40),histtype='bar', ec='k') # histogramme creation

pygo.auxCamera.setExposureTime(10000) # for 10000ms
fra= pygo.auxCamera.getFutureFrames(10, timeoutSec=200).toNumpyArray()
bias10000ms=np.median(fra, axis=2) # median according to the 3rd dimension
pyfits.writeto(os.path.join(baseDir, 'dark_10000ms.fits'), bias10000ms) #write
biasv10000ms=np.std(fra, axis=2) # variance according to the 3rd dimension
pyfits.writeto(os.path.join(baseDir, 'dark_10000ms_std.fits'), biasv10000ms)
n, bins, patches= plt.hist(GAIN_E_PER_ADU*biasv10000ms.flatten(),\50,
range=(0,40),histtype='bar', ec='k') # histogramme creation
```

```
        pygo.auxCamera.setExposureTime(20000) # for 20000ms
        fra=np.zeros((256,256,10))
        for i in range(10):
                fra[:,:,i]=pygo.auxCamera.getFutureFrames(1,
                timeoutSec=40).toNumpyArray()
        bias20000ms=np.median(fra, axis=2) # median according to the 3rd dimension
        pyfits.writeto(os.path.join(baseDir, 'dark_20000ms.fits'), bias20000ms)
        biasv20000ms=np.std(fra, axis=2) # variance according to the 3rd dimension
        pyfits.writeto(os.path.join(baseDir, 'dark_20000ms_std.fits'), biasv20000ms)
        n, bins, patches= plt.hist(GAIN_E_PER_ADU*biasv20000ms.flatten(),\50,
        range=(0,40),histtype='bar', ec='k') # histogramme creation

# Determination of bad pixels
def getMappaBadFromMedianAndStdFrames(self, bias, biasv, tit):
        from scipy.ndimage.filters import gaussian_filter # import library
        filbias= gaussian_filter(bias, 5) # Gauss filter application
        mappaHot= (bias>3*np.std(filbias)+ filbias)|(bias<1.) # erroneous by heat
        mappaDancer= biasv>3*np.std(biasv) + filbias # erroneous by their volatility
        mappaBad= mappaHot+ mappaDancer # wrong pixels = sum of erroneous one
        imshow(mappaBad, origin='lower')
        title('Bad pixels fraction= %.2f%%, '
        %(np.sum(mappaBad)/(mappaBad.shape[0]*mappaBad.shape[1])*100)+tit)
        # image title
        print(np.sum(mappaBad)/(mappaBad.shape[0]*mappaBad.shape[1])*100)
        # displays the image of the wrong pixels so their positions on the sensor
```

Annexe 4 : Code in Python language used to determine read noise and dark current

```
# This program evolved during the measurements, before the discovery of the technical program, I was using more data and had not yet calculated the confidence interval of the data for the dark current. There were only other loops that processed the wrong images.
# import librairies
import astropy.io.fits
import numpy as np
import matplotlib.pyplot as plt
# variables
couple_b=[] # variance for each successive image pair
med_b=[] # average of each image pairs successives de b
med=[] # average of each pixel in the image
difference=[] # b-a
couple_s=[] # signal - b for measurements dedicated to reading noise
s=[] # average couples_s per pixel
med_s=[] # average of the value of s on all pixels
couple_var=[] # variance for an image pair
var=[] # time average of the variances of the image pairs
med_var=[] # spatial mean of the time averages of the variances of the image pairs
med_med=[] # spatial average of image pair time averages
med_s=[] # spatial average of the temporal variance averages of image pair variances for integration time
erreur=[] # due to noisy data, 99.7% confidence interval of a Gaussian curve
t=[] # time in  sec
te=[] # time in sec for the error
a=0 # first image of the couple
b=0 # second image of the couple
n=0 # counter
x=0 # scalare representing the spatial average of the time averages for the integration time k
y=0 # scalare representing the spatial average of s' for the integration time k
z=0 # scalare representing the spatial mean of the temporal variances of the image pair variances for the integration time k
```

```
# calcul of b
for i in range(1,11): # for b, an integration time of 1s
	name=str(i)+'.fits' # will process the images of 1.fits to 10.fits
	n+=1 # the counter is incremented
	if n==1: # if n=1
		a=astropy.io.fits.getdata(name) # extracts the information from the image a
	else:	# n=2
		b=astropy.io.fits.getdata(name) # extracts the information from the image b
		n=0 # n becomes 0 again because we have 2 images
		couple_b.append(b-a) #calculation of the difference of a pair of images
		couple_s.append(a+b) #calculation of s' of a pair of images
		couple_var.append((b-a)**2) #calculation of the temporal variance of the pair
m=0 #average of the temporal variance of each image pair
p=0 #average of each (image pair - b)
q=0 # variance of each image pair
t.append(1) # it is indicated that the measurements were made for an integration time of 1s
for i in range(len(couple_b)): # calculate the average values of all the pairs
	m+=couple_b[i] # the values are added together
	p+=couple_s[i] # the values are added together
	q+=couple_var[i] # the values are added together
m=m/len(couple_b) # we divide by the total number of values
p=p/len(couple_b) # we divide by the total number of values
q=q/len(couple_b) # we divide by the total number of values
med_b.append(m) # addition of m
med.append(m) # addition of m
s.append(p/2) # addition of p
var.append(q) # addition of q
te.append(1) #lower limit error
te.append(1) #upper error limit
# measure of the dark, you don't remove the background!
for i in range(21,31): # for 2s
	name=str(i)+'.fits' # will process the images of 21.fits to 30.fits
	n+=1 # the counter is incremented
	if n==1: # if n=1
		a=astropy.io.fits.getdata(name) # extracts the information from the image a
	else:	# n=2
		b=astropy.io.fits.getdata(name) # extracts the information from the image b
		n=0 # n becomes 0 again because we have 2 images
		couple_b.append(b-a) #calculation of the difference of a pair of images
		couple_s.append(a+b) #calculation of s' of a pair of images
		couple_var.append((b-a)**2) #calculation of the temporal variance of the pair
m=0 # average of the temporal variance of each image paire
p=0 # average of each (image pair - b)
q=0 #  variance of each image pair
t.append(2) # it is indicated that the measurements were made for an integration time ofit is
indicated that the measurements were made for an integration time of 2s
```

```
for i in range(5): # calculate the average values of all the pairs
        m+=couple_b[i] # the values are added together
        p+=couple_s[i] # the values are added together
        q+=couple_var[i] # the values are added together
m=m/len(couple_b) # we divide by the total number of values
p=p/len(couple_b) # we divide by the total number of values
q=q/len(couple_b) # we divide by the total number of values
med.append(m) # addition of m
s.append(p/2-b) # addition of p
var.append(q) # addition of q
te.append(2) #lower limit error
te.append(2) #upper error limit

for i in range(11,21): # for 5s
        name=str(i)+'.fits' # will process the images of 11.fits to 20.fits
        n+=1 # the counter is incremented
        if n==1: # if n=1
                a=astropy.io.fits.getdata(name) # extracts the information from the image a
        else:   # n=2
                b=astropy.io.fits.getdata(name) # extracts the information from the image b
                n=0 # n becomes 0 again because we have 2 images
                couple_b.append(b-a) #calculation of the difference of a pair of images
                couple_s.append(a+b) #calculation of s' of a pair of images
                couple_var.append((b-a)**2) #calculation of the temporal variance of the pair
m=0 #average of each image pair
p=0 #average of each (image pair - b)
q=0 # variance of each image pair
t.append(5) # it is indicated that the measurements were made for an integration time of 5s
for i in range(5,10): # calculate the average values of all the pairs
        m+=difference[i] # the values are added together
        p+=couple_s[i] # the values are added together
        q+=couple_var[i] # the values are added together
m=m/5 # we divide by the total number of values
p=p/5 # we divide by the total number of values
q=q/5 # we divide by the total number of values
med.append(m) # addition of m
s.append(p/2) # addition of p
var.append(q) # addition of q
te.append(5) #lower limit error
te.append(5) #upper error limit
```

```
for i in range(31,41): # for 10s
        name=str(i)+'.fits' # will process the images of 31.fits to 40.fits
        n+=1 # the counter is incremented
        if n==1: # if n=1
                a=astropy.io.fits.getdata(name) # extracts the information from the image a
        else:   # n=2
                b=astropy.io.fits.getdata(name) # extracts the information from the image b
                n=0 # n becomes 0 again because we have 2 images
                couple_b.append(b-a) #calculation of the difference of a pair of images
                couple_s.append(a+b) #calculation of s' of a pair of images
                couple_var.append((b-a)**2) #calculation of the temporal variance of the pair
m=0 #average of each image pair
p=0 #average of each (image pair - b)
q=0 # variance of each image pair
t.append(10) # integration time of 10s
for i in range(10,15): # calculate the average values of all the pairs
        m+=difference[i] # the values are added together
        p+=couple_s[i] # the values are added together
        q+=couple_var[i] # the values are added together
m=m/5 # we divide by the total number of values
p=p/5 # we divide by the total number of values
q=q/5 # we divide by the total number of values
med.append(m) # addition of m
s.append(p/2) # addition of p
var.append(q) # addition of q
te.append(10) #lower limit error
te.append(10) #upper error limit

for i in range(41,51): # for 15s
name=str(i)+'.fits' # will process the images of 41.fits to 50.fits
        n+=1 # the counter is incremented
        if n==1: # if n=1
                a=astropy.io.fits.getdata(name) # extracts the information from the image a
        else:   # n=2
                b=astropy.io.fits.getdata(name) # extracts the information from the image b
                n=0 # n becomes 0 again because we have 2 images
                couple_b.append(b-a) #calculation of the difference of a pair of images
                couple_s.append(a+b) #calculation of s' of a pair of images
                couple_var.append((b-a)**2) #calculation of the temporal variance of the pair
m=0 #average of each image pair
p=0 #average of each (image pair - b)
q=0 # variance of each image pair
t.append(15) # integration time of 15s
for i in range(15,20): # calculate the average values of all the pairs
        m+=difference[i] # the values are added together
        p+=couple_s[i] # the values are added together
        q+=couple_var[i] # the values are added together
```

```
m=m/5 # we divide by the total number of values
p=p/5 # we divide by the total number of values
q=q/5 # we divide by the total number of values
med.append(m) # addition of m
s.append(p/2) # addition of p
var.append(q) # addition of q
te.append(15) #lower limit error
te.append(15) #upper error limit

for i in range(51,61): # for 20s
        name=str(i)+'.fits' # will process the images of 51.fits to 60.fits
        n+=1 # the counter is incremented
        if n==1: # if n=1
                a=astropy.io.fits.getdata(name) # extracts the information from the image a
        else:   # n=2
                b=astropy.io.fits.getdata(name) # extracts the information from the image b
                n=0 # n becomes 0 again because we have 2 images
                couple_b.append(b-a) #calculation of the difference of a pair of images
                couple_s.append(a+b) #calculation of s' of a pair of images
                couple_var.append((b-a)**2) # calculation of the temporal variance of the pair
m=0 #average of each image pair
p=0 #average of each (image pair - b)
q=0 # variance of each image pair
t.append(20) # integration time of 20s
for i in range(20,25): # calculate the average values of all the pairs
        m+=difference[i] # the values are added together
        p+=couple_s[i] # the values are added together
        q+=couple_var[i] # the values are added together
m=m/5 # we divide by the total number of values
p=p/5 # we divide by the total number of values
q=q/5 # we divide by the total number of values
med.append(m) # addition of m
s.append(p/2) # addition of p
var.append(q) # addition of q
te.append(20) #lower limit error
te.append(20) #upper error limit
for i in range(61,71): # for 25s
        name=str(i)+'.fits' # will process the images of 60.fits to 70.fits
        n+=1 # the counter is incremented
        if n==1: # if n=1
                a=astropy.io.fits.getdata(name) # extracts the information from the image a
        else:   # n=2
                b=astropy.io.fits.getdata(name) # extracts the information from the image b
                n=0 # n becomes 0 again because we have 2 images
                couple_b.append(b-a) #calculation of the difference of a pair of images
                couple_s.append(a+b) #calculation of s' of a pair of images
                couple_var.append((b-a)**2) #calculation of the temporal variance of the pair
```

```
m=0 #average of each image pair
p=0 #average of each (image pair - b)
q=0 # variance of each image pair
t.append(25) # integration time of 25s
for i in range(25,30):  # calculate the average values of all the pairs
        m+=difference[i] # the values are added together
        p+=couple_s[i] # the values are added together
        q+=couple_var[i] # the values are added together
m=m/5 # we divide by the total number of values
p=p/5 # we divide by the total number of values
q=q/5 # we divide by the total number of values
med.append(m) # addition of m
s.append(p/2) # addition of p
var.append(q) # addition of q
te.append(25) # lower limit error
te.append(25) # upper error limit

for i in range(101,111): # for 30s
        name=str(i)+'.fits' # will process the images of 101.fits to 110.fits
        n+=1 # the counter is incremented
        if n==1: # if n=1
                a=astropy.io.fits.getdata(name) # extracts the information from the image a
        else:   # n=2
                b=astropy.io.fits.getdata(name) # extracts the information from the image b
                n=0 # n becomes 0 again because we have 2 images
                couple_b.append(b-a) #calculation of the difference of a pair of images
                couple_s.append(a+b) #calculation of s' of a pair of images
                couple_var.append((b-a)**2) #calculation of the temporal variance of the pair
m=0 #average of each image pair
p=0 #average of each (image pair - b)
q=0 # variance of each image pair
t.append(30) # integration time of 30s
for i in range(30,35): # calculate the average values of all the pairs
        m+=difference[i] # the values are added together
        p+=couple_s[i] # the values are added together
        q+=couple_var[i] # the values are added together
m=m/5 # we divide by the total number of values
p=p/5 # we divide by the total number of values
q=q/5 # we divide by the total number of values
med.append(m) # addition of m
s.append(p/2) # addition of p
var.append(q) # addition of q
te.append(30) #lower limit error
te.append(30) #upper error limit
```

```
for i in range(111,121): # for 35s
        name=str(i)+'.fits' # will process the images of 111.fits to 121.fits
        n+=1 # the counter is incremented
        if n==1: # if n=1
                a=astropy.io.fits.getdata(name) # extracts the information from the image a
        else:   # n=2
                b=astropy.io.fits.getdata(name) # extracts the information from the image b
                n=0 # n becomes 0 again because we have 2 images
                couple_b.append(b-a) #calculation of the difference of a pair of images
                couple_s.append(a+b) #calculation of s' of a pair of images
                couple_var.append((b-a)**2) #calculation of the temporal variance of the pair
m=0 #average of each image pair
p=0 #average of each (image pair - b)
q=0 # variance of each image pair
t.append(35) # integration time of 35s
for i in range(35,40): # calculate the average values of all the pairs
        m+=difference[i] # the values are added together
        p+=couple_s[i] # the values are added together
        q+=couple_var[i] # the values are added together
m=m/5 # we divide by the total number of values
p=p/5 # we divide by the total number of values
q=q/5 # we divide by the total number of values
med.append(m) # addition of m
s.append(p/2) # addition of p
var.append(q) # addition of q
te.append(35) #lower limit error
te.append(35) #upper error limit

for i in range(121,131): # for 40s
        name=str(i)+'.fits' # will process the images of 121.fits to 130.fits
        n+=1 # the counter is incremented
        if n==1: # if n=1
                a=astropy.io.fits.getdata(name) # extracts the information from the image a
        else:   # n=2
                b=astropy.io.fits.getdata(name) # extracts the information from the image b
                n=0 # n becomes 0 again because we have 2 images
                couple_b.append(b-a) # calculation of the difference of a pair of images
                couple_s.append(a+b) # calculation of s' of a pair of images
                couple_var.append((b-a)**2) # calculation of the temporal variance of the pair
m=0 #average of each image pair
p=0 #average of each (image pair - b)
q=0 # variance of each image pair
t.append(40) # integration time of 40s
for i in range(40,45): # calculate the average values of all the pairs
        m+=difference[i] # the values are added together
        p+=couple_s[i] # the values are added together
        q+=couple_var[i] # the values are added together
```

```
m=m/5 # we divide by the total number of values
p=p/5 # we divide by the total number of values
q=q/5 # we divide by the total number of values
med.append(m) # addition of m
s.append(p/2) # addition of p
var.append(q) # addition of q
te.append(40) #lower limit error
te.append(40) #upper error limit

for i in range(131,141): # for 45s
	name=str(i)+'.fits' # will process the images of 131.fits to 140.fits
	n+=1 # the counter is incremented
	if n==1: # if n=1
		a=astropy.io.fits.getdata(name) # extracts the information from the image a
	else:	# n=2
		b=astropy.io.fits.getdata(name) # extracts the information from the image b
		n=0 # n becomes 0 again because we have 2 images
		couple_b.append(b-a) #calculation of the difference of a pair of images
		couple_s.append(a+b) #calculation of s' of a pair of images
		couple_var.append((b-a)**2) #calculation of the temporal variance of the pair
m=0 #average of each image pair
p=0 #average of each (image pair - b)
q=0 # variance of each image pair
t.append(45) # integration time of 45s
for i in range(45,50): # calculate the average values of all the pairs
	m+=difference[i] # the values are added together
	p+=couple_s[i] # the values are added together
	q+=couple_var[i] # the values are added together
m=m/5 # we divide by the total number of values
p=p/5 # we divide by the total number of values
q=q/5 # we divide by the total number of values
med.append(m) # addition of m
s.append(p/2) # addition of p
var.append(q) # addition of q
te.append(45) #lower limit error
te.append(45) #upper error limit

for i in range(71,81): # for 50s
	name=str(i)+'.fits' # will process the images of 70.fits to 80.fits
	if n==1: # if n=1
		a=astropy.io.fits.getdata(name) # extracts the information from the image a
	else:	# n=2
		b=astropy.io.fits.getdata(name) # extracts the information from the image b
		n=0 # n becomes 0 again because we have 2 images
		couple_b.append(b-a) #calculation of the difference of a pair of images
		couple_s.append(a+b) #calculation of s' of a pair of images
		couple_var.append((b-a)**2) #calculation of the temporal variance of the pair
```

```
m=0 #average of each image pair
p=0 #average of each (image pair - b)
q=0 # variance of each image pair
t.append(50) # integration time of 50s
for i in range(50,55): # calculate the average values of all the pairs
        m+=difference[i] # the values are added together
        p+=couple_s[i] # the values are added together
        q+=couple_var[i] # the values are added together
m=m/5 # we divide by the total number of values
p=p/5 # we divide by the total number of values
q=q/5 # we divide by the total number of values
med.append(m) # addition of m
s.append(p/2) # addition of p
var.append(q) # addition of q
te.append(50) #lower limit error
te.append(50) #upper error limit
# calculations of spatial averages
for k in range(len(med)): # integration time k
        for i in range(2048): # 2048: number of abscissa
                for j in range(2040): # 2040: number of ordinates
                        x+=med[k][i,j] # the values are added together de chaque pixel
                        y+=s[k][i,j] # the values are added together de chaque pixel
                        z+=var[k][i,j] # the values are added together de chaque pixel
        x=x/(2040*2048) # divide by the total number of pixels
        y=y/(2040*2048) # divide by the total number of pixels
        z=z/(2040*2048) # divide by the total number of pixels
        erreur.append(x-3*z/10) #lower limit error
        erreur.append(x+3*z/10) #upper error limit
        med_med.append(x) # addition of x
        med_s.append(y) # addition of y
        med_var.append(z) #addition of z
plt.plot(t,med_med,'o') # the spatial average of the time averages is plotted as a function of
each time interval to derive the obscure current
plt.plot(te,erreur,'o') # the 99.7% confidence interval is plotted
plt.xlabel("integration time (sec)") # title of the abscissa
plt.ylabel('spatial average of time averages') # title of the ordinates
plt.title('spatial average of time averages as a function of time (sec)') # graphic title
plt.show()
```

```
for i in range(81,91): # images for 1s acquisition time and a brightness level
        name=str(i)+'.fits' # will process the images of 80.fits to 90.fits
        if n==1: # if n=1
                a=astropy.io.fits.getdata(name) # extracts the information from the image a
        else:  # n=2
                b=astropy.io.fits.getdata(name) # extracts the information from the image b
                n=0 # n becomes 0 again because we have 2 images
                couple_b.append(b-a) #calculation of the difference of a pair of images
                couple_s.append(a+b) #calculation of s' of a pair of images
                couple_var.append((b-a)**2) #calculation of the temporal variance of the pair
m=0 #average of each image pair
p=0 #average of each (image pair - b)
q=0 # variance of each image pair
for i in range(55,60): # calculate the average values of all the pairs
        m+=difference[i] # the values are added together
        p+=couple_s[i] # the values are added together
        q+=couple_var[i] # the values are added together
m=m/5 # we divide by the total number of values
p=p/5 # we divide by the total number of values
q=q/5 # we divide by the total number of values
med.append(m) # addition of m
s.append(p/2-med_b) # addition of p
var.append(q) # addition of q

for i in range(91,101): # images for 1s acquisition time and a brightness level
        name=str(i)+'.fits' # will process the images of 90.fits to 100.fits
        if n==1: # if n=1
                a=astropy.io.fits.getdata(name) # extracts the information from the image a
        else:   # n=2
                b=astropy.io.fits.getdata(name) # extracts the information from the image b
                n=0 # n becomes 0 again because we have 2 images
                couple_b.append(b-a) #calculation of the difference of a pair of imagess
                couple_s.append(a+b) #calculation of s' of a pair of images
                couple_var.append((b-a)**2) #calculation of the temporal variance of the pair
m=0 #average of each image pair
p=0 #average of each (image pair - b)
q=0 # variance of each image pair
for i in range(60,65): # calculate the average values of all the pairs
        m+=difference[i] # the values are added together
        p+=couple_s[i] # the values are added together
        q+=couple_var[i] # the values are added together
m=m/5 # we divide by the total number of values
p=p/5 # we divide by the total number of values
q=q/5 # we divide by the total number of values
med.append(m) # addition of m
s.append(p/2-med_b) # addition of p
var.append(q) # addition of q
```

```
plt.plot(med_s, med_var,'o') # the spatial mean of the temporal variances of image pairs for each integration time is plotted as a function of the spatial mean of s' for each time interval to derive from it the reference coefficient corresponding to the gain and the ordinate at the origin corresponding to the reading noise.
plt.xlabel("spatial mean of s'") # title of the abscissa
plt.ylabel("spatial mean of temporal variances") # title of the ordinates
plt.title('graph to determine gain and reading noise') # graphic title
plt.show()
```

Abstract

I have discovered the social climate of a laboratory and obtained international experience. The use of the English language was necessary in order to communicate and I have learnt some basis of Italian language, this was interesting. Moreover, the aim of this internship was to develop my skills in optics and in data processing through many manipulations on instruments of measure. This was the case through the manipulation of the camera and with the explanations supplied by my tutor, Dr Armando RICCARDI, about adaptive optics and about the sources of measure errors. I also improved my python skills and discovered the astropy library.